Homework — 1.4.1 Data Types — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Total: Part A 14 + Part B 6 = 20 marks.

Part A

1. [2] 156 → 8-bit binary. 156 = 128 + 16 + 8 + 4. Place values: 128(1) 64(0) 32(0) 16(1) 8(1) 4(1) 2(0) 1(0). Check: 128 + 16 + 8 + 4 = 156. ✓ Answer: **`10011100`**. *Mark: correct subtraction/method showing the place values used (1); correct 8-bit answer `10011100` (1).*

2. [2] `11001010` → hex. Split into nibbles from the right: `1100 1010`. `1100` = 8 + 4 = 12 = **C**. `1010` = 8 + 2 = 10 = **A**. Answer: **`CA`**. *Mark: correct nibble split and values 12 and 10 (1); correct hex `CA` (1).*

3. [2] `6F` → denary. 6 = 6, F = 15. (6 × 16) + 15 = 96 + 15 = **111**. *Mark: 6 × 16 = 96 (1); + 15 = 111 (1).*

4. [4] 52 − 37 = 52 + (−37). +52 = `00110100` (32 + 16 + 4 = 52). ✓ +37 = `00100101` (32 + 4 + 1 = 37). ✓ −37: invert `00100101` → `11011010`, then + 1 → `11011011`. Add:

```
  00110100   (52)
+ 11011011   (−37)
-----------
1 00001111
```

The carry out of the MSB is **discarded** (the leading 1 is dropped), leaving `00001111`. `00001111` = 8 + 4 + 2 + 1 = **15**. ✓ (52 − 37 = 15) *Mark: +52 and +37 correct in binary (1); correct two's complement of 37 = `11011011` (1); correct binary addition giving `1 00001111` (1); states final carry is discarded → answer 15 (1).*

5. [2] `01001100` arithmetic right shift 2 places. The MSB (sign bit) is **0**, so copies of 0 are brought in from the left; bits move right two places and the two rightmost bits are lost: `01001100` → (1 right) `00100110` → (2 right) `00010011`. Answer: **`00010011`**. Effect: divided by 2² = **÷4** (integer division). Check: `01001100` = 76; `00010011` = 16 + 2 + 1 = 19 = 76 ÷ 4. ✓ *Mark: correct shifted pattern `00010011` (1); states effect = ÷4 / ÷2² (1).*

6. [2] Any one developed reason, e.g.: - Integers are stored exactly as whole binary numbers, whereas reals/floats use **floating-point** (mantissa × 2^exponent) which can only **approximate** many values, causing rounding errors (1) — so using a real where an exact whole number (e.g. a loop counter or array index) is needed could give wrong results or fail (1). - (Alternatively: data type fixes the range/precision and the operations allowed, so the wrong type wastes memory or cannot store the required values.) *Mark: a valid representational point about integer vs real storage (1); consequence of choosing wrongly (1). Max 2.*

Part B

7. [1] The MAR holds the **address** of the memory location that is about to be read from or written to (e.g. the address copied from the PC during fetch). *(AO1)*

8. [3] Virtual memory uses an area of **secondary storage** (a swap/page file on disk) and treats it **as if it were RAM** (1). When RAM is full, pages not currently in use are **swapped out to disk**, and pages that are needed are **swapped back into RAM** when accessed (a page fault triggers the swap) (1). This lets the program (or more programs at once) run even though it is larger than the physical RAM, though it is **slower** than real RAM (1). *(AO1)*

9. [2] Difference: **lossless** compression removes no data and the original file can be reconstructed **exactly**; **lossy** compression permanently discards some data so the original **cannot** be perfectly recovered (1). Uses: lossless for text/program code/ZIP/PNG (where exactness matters); lossy for images/audio/video such as JPEG or MP3 (where small quality loss is acceptable for smaller files) (1). *(AO1)*